Part 6: Finishing the Self-Attention Sublayer

These notes describe the layer-0 attention implementation shown in Part 6, before the later MLP and decode changes. The source links to `src/main.cpp` and `src/kernels.cu` now point to the newer implementation; use `git show ec066a7:src/main.cpp` and `git show ec066a7:src/kernels.cu` for the Part 6 snapshot. The "Attention" section of `notes.md` gives the working explanation. Start with the Q/K/V projection notes if the three projected buffers are still unfamiliar.

What we are finishing

Now we have `Q`, `K`, and `V` for every prompt token. RoPE has rotated Q and K. How do we get from those vectors to an updated hidden state? This is the big attention calculation people talk about:

```
Q and K -> scaled token-to-token scores
        -> causal mask -> stable softmax -> attention probabilities
probabilities and V -> one weighted value mixture per query head
32 head outputs -> concatenate -> learned output projection
old hidden state + attention update -> new hidden state
```

That sequence completes the **self-attention sublayer** for layer 0 at this point in the series. It does not yet complete the whole Llama decoder layer: the post-attention RMSNorm, gated MLP, and second residual are still to come. Nor does the Part 6 executable run all 16 layers or produce a next token yet.

Why we needed attention in the first place

My notes use "I just clean windows" versus "I just downloaded Windows." The surrounding words help decide what a token is talking about. Attention gives a token a way to pull in information from other tokens in the prompt. In this causal model, a token may use **itself and earlier tokens**, not future ones.

The query/key/value analogy is still useful: a query asks what could help, keys say what can be matched, and values are what gets passed along. But attention weights are learned numerical behavior, not a hand-written English dictionary. A particular head might learn a useful relation, but we do not manually designate "the Windows meaning head."

The full math for one query head

Let `T` be the prompt length. One query head has `Q_h [T, 64]`; the K and V head shared with it each have `[T, 64]`. For query head `h`, let `g = h / 4` be its K/V head. Then:

```
S_h[i,j] = dot(rotated_Q_h[i], rotated_K_g[j]) / sqrt(64) + mask[i,j]
mask[i,j] = 0 if j <= i, and -infinity if j > i

P_h[i,j] = exp(S_h[i,j] - max_allowed_score_in_row)
           / sum_k exp(S_h[i,k] - max_allowed_score_in_row)

head_output_h[i] = sum_j P_h[i,j] * V_g[j]
```

The complete head output is `[T, 64]`. We place 32 of those 64-value chunks beside each other to get `[T, 2048]`, multiply by the learned output matrix `W_o`, and add the result to the incoming hidden state.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The rough notes wrote the score and mask in two slightly different orders. The code does the **Q·K dot product, scales by** `1 / sqrt(64) = 1/8`**, and then masks future positions**. If an entry is masked to `-infinity`, softmax gives it probability zero. We must not divide a finite mask value by the scale or let a future entry participate in the row maximum.

One worked three-token example

Let's use one tiny fake head and pretend the scores have **already been scaled**. Rows are query positions; columns are key positions. Before masking:

```
             key 0   key 1   key 2
query 0        1       4       3
query 1        2       1       5
query 2        0       1       2
```

The numbers in the upper-right triangle are tempting, but they are future tokens for those queries. Mask them first:

```
             key 0   key 1   key 2
query 0        1      -inf    -inf
query 1        2       1      -inf
query 2        0       1       2
```

Stable softmax turns **each row independently** into probabilities:

```
query 0 -> [1.000, 0.000, 0.000]
query 1 -> [0.731, 0.269, 0.000]
query 2 -> [0.090, 0.245, 0.665]
```

For query 1, subtract the row max `2`, exponentiate `[0, -1, -inf]` to `[1, 0.3679, 0]`, then divide by the sum `1.3679`. That is where `0.731` and `0.269` come from. The mask is about **permission to see a token**; the probability measures **how much its value contributes** among allowed tokens.

Now give the three key positions toy value vectors `V_0 = [10, 0]`, `V_1 = [0, 20]`, and `V_2 = [5, 5]`. Query 1's mixed result is:

```
0.731 * [10, 0] + 0.269 * [0, 20] + 0 * [5, 5]
    ≈ [7.31, 5.38]
```

We have taken information from the **values**, using weights determined by Q and K. The numbers above are teaching examples, not output from this model. Real heads have 64-value vectors, 32 query heads, and BF16 rounding at several storage boundaries.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The actual shapes and GQA mapping

For this Llama 3.2 1B configuration:

Buffer	Logical shape	What it contains
`q_gpu`	`[T, 32, 64]`, flattened as `[T, 2048]`	Rotated query heads.
`k_gpu`	`[T, 8, 64]`, flattened as `[T, 512]`	Rotated key heads.
`v_gpu`	`[T, 8, 64]`, flattened as `[T, 512]`	Unrotated value heads.
`attention_scores_gpu`	`[32, T, T]`	First scaled scores, then masked scores, then probabilities **in the same allocation**.
`attention_output_gpu`	`[T, 2048]`	The 32 mixed head outputs concatenated by where they are written.
`output_projection_gpu`	`[T, 2048]`	Learned output projection, ready for residual addition.

Head mapping is `kv_head_index = query_head_index / 4`: Q heads 0-3 share K/V head 0, and so on up to Q heads 28-31 sharing K/V head 7. Each Q head still has its **own** score and probability matrix. Sharing K/V heads reduces the amount of K/V data; it does not mean all four query heads must look at the same token.

The score tensor's storage grows as `32 × T × T`. At `T = 2048`, BF16 scores alone occupy `32 × 2048² × 2 = 268,435,456` bytes, or 256 MiB. This straightforward prefill path materializes those matrices. A later optimized attention implementation could avoid some of that memory, but this code is useful because every step is visible.

Step 1: scaled Q·K scores with cuBLAS

`computeGqaAttentionScores` loops over all 32 query heads. For head `h`, it takes a pointer to that 64-value Q slice and a pointer to K head `h / 4`. The vectors are physically inside token-major rows: Q rows have stride `2048`, K rows have stride `512`. Passing `q + h * 64` or `k + (h/4) * 64` changes the **head offset**, but not the distance to the next token's row.

For a query token `i` and key token `j`, the desired score is:

```
score[h, i, j] = sum over d=0..63 of Q[i,h,d] * K[j,h/4,d] / 8
```

`alpha = 1 / sqrt(HEAD_DIM)` supplies that `/ 8` during `cublasGemmEx`; `beta = 0` means the old score buffer is ignored. The GEMM's logical result for each head is `[T, T]`. Because the buffers are row-major but classic cuBLAS uses a column-major view, the call uses `CUBLAS_OP_T` for K and `CUBLAS_OP_N` for Q. `lda = KV_DIM` and `ldb = HIDDEN_SIZE` preserve the full token strides even though we selected only 64 columns of each row. `ldc = T` makes each head's score matrix contiguous.

One clarification from my rough notes: negative dot products are perfectly valid scores. We do **not** run softmax because negative numbers are somehow meaningless. We run it because attention needs nonnegative weights, normalized **across the allowed key positions** for each query row. Scaling first also keeps score magnitudes from growing with the 64 summed dimensions and pushing softmax too sharply toward one key.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

cuBLAS takes BF16 Q/K values, uses `CUBLAS_COMPUTE_32F`, then stores BF16 scores. FP32 compute helps the dot product, but storing scores as BF16 still rounds them before the next kernel.

Step 2: make future positions impossible to use

`causalMaskKernel` launches one thread per score element in flattened `[head, query, key]` storage. It recovers the two token indices with:

```
key_token = score_index % token_count;
query_token = (score_index / token_count) % token_count;
```

When `key_token > query_token`, the thread writes negative infinity. The diagonal remains valid, so every query row always has **at least its own token** to attend to. This is why the later softmax always has at least one finite candidate. A query at position 0 may use only key 0; a query at position 2 may use keys 0, 1, and 2.

The score matrix is not symmetric after masking. That is expected: the token at position 2 can use position 0, while position 0 cannot peek at position 2. RoPE encodes position in Q/K; the causal mask enforces this one-way information flow.

Step 3: stable softmax, one row at a time

`stableSoftmaxKernel` launches **one CUDA block per** `(query head, query token)` **row**, with 256 threads in that block. Every thread visits key positions `thread_index`, `thread_index + 256`, and so on. That stride lets the same kernel process a row longer than 256 keys.

It makes two shared-memory reductions:

1. Find the **maximum** score in the row, ignoring nothing explicitly because masked values are already `-infinity`.
2. Sum `exp(score - row_max)` across all key positions.

Then each thread writes `exp(score - row_max) / sum` to the matching location in the same `scores` buffer. Subtracting the maximum does not change softmax probabilities, because the same constant is subtracted from every allowed score, but it prevents very large positive scores from overflowing `exp`.

The reductions and exponentials use FP32. The probabilities are stored as BF16, so their stored row sum may be **approximately**, rather than exactly, 1. A masked `-infinity` gives `exp(-infinity) = 0`, so future positions have zero probability. `launchCausalMask` runs before `launchStableSoftmax` in `prefill`; reversing those calls would be wrong.

Step 4: use probabilities to mix V

`mixGqaValues` loops over 32 query heads. It selects that head's `[T, T]` probability matrix and its shared V head `h / 4`, then performs a cuBLAS matrix multiplication:

```
head_output_h [T, 64] = P_h [T, T] × V_(h/4) [T, 64]
```

For one query token `i` and one output feature `d`, that is `sum_j P_h[i,j] * V_(h/4)[j,d]`. Future `j` values contribute nothing because their probabilities are zero. Values are **not** passed through RoPE: Q/K choose what to take; V supplies what gets taken.

Again, the cuBLAS arguments look strange until you remember the row-major storage. Both operands use `CUBLAS_OP_N` in cuBLAS's column-major view to produce the flat bytes of the wanted row-major `P × V`. The V

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

token stride is still `KV_DIM = 512`; each probability row has stride `T`; and the destination token stride is `HIDDEN_SIZE = 2048`.

The code points `output_head` at `attention_output_gpu + h * 64`. That is how the 32 results land side by side inside each token's 2,048-value row. We do not need a separate concatenate kernel:

```
token i output row = [head 0's 64 values | head 1's 64 values | ... | head 31's 64 values]
```

Step 5: output projection and residual connection

The concatenated head output is not yet the attention update in the model's hidden space. `prefill` calls the same `projectBf16RowMajor` helper used for Q/K/V, this time with `weights.w_o[0]` and output width `2048`:

```
attention_update = attention_output × Woᵀ     [T, 2048]
```

`W_o` is learned. It mixes features from the different heads and produces a vector with the same width as the incoming hidden state, so they can be added. The CUDA `residualAddKernel` gives one thread each hidden value, converts the two BF16 inputs to `float`, adds them, and writes BF16 back **in place**:

```
activations_gpu[token, feature] += output_projection_gpu[token, feature]
```

My Δx picture from the rough notes is good here: the attention path computes an update, and the residual adds it to the old vector. The subtle part is **which old vector**. This is a pre-norm layer: RMSNorm made `normalized_gpu` for Q/K/V, but the skip path uses the original layer input in `activations_gpu`. For layer 0 that input came from embedding gather. In later layers it would be the previous layer's hidden state. Adding to `normalized_gpu` instead would implement a different computation.

How this appears in `prefill`

The source order is worth tracing with your finger:

```
embedding gather -> activations_gpu
RMSNorm -> normalized_gpu
project Wq, Wk, Wv -> q_gpu, k_gpu, v_gpu
RoPE -> modifies q_gpu and k_gpu
computeGqaAttentionScores -> attention_scores_gpu contains scaled scores
launchCausalMask -> future scores become -infinity
launchStableSoftmax -> same buffer now contains probabilities
mixGqaValues -> attention_output_gpu
project Wo -> output_projection_gpu
launchResidualAdd -> modifies activations_gpu
```

The host checks cuBLAS statuses and CUDA launch results along the way, then synchronizes to catch GPU execution errors. The temporary buffers are freed at the end of `prefill`. In the Part 6 version, the updated `activations_gpu` is also freed rather than passed into a post-attention norm or another layer. That version demonstrates this attention computation, but does not yet generate text.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

What still makes a full Transformer layer

After the attention residual, Llama's decoder layer continues with a second RMSNorm and a gated feed-forward network: gate and up projections, SiLU on the gate, elementwise gate/up multiplication, down projection, and another residual add. Then the next layer repeats the whole sequence. The Part 6 source has pointers to those MLP weights but does not execute that path. Part 7 completes it and predicts the first next token.

If you remember one line from this note, use this one:

```
Attention = use rotated Q/K to decide *where to look*,
            use probabilities to mix V from those positions,
            project the mixture, then add it to the incoming hidden state.
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.